

EEE 495 Electrical and Electronics Engineering Design II

Final Report



Project Title / Product Name: FarmGate Sentinel

Date: December 29, 2025

Team No: 2

Student ID	Name and Surname	e-mail
21802551	Muhammed Gölcük	muhammed.golcuk@ug.bilkent.edu.tr
21903121	Yusuf Semih Gürkök	semih.gurkok@ug.bilkent.edu.tr
22002782	Zeynep İrem İpek	irem.ipek@ug.bilkent.edu.tr
22001648	İsmail Özgenç	ismail.ozgenc@ug.bilkent.edu.tr

Department of Electrical and Electronics Engineering
Bilkent University
Ankara, Türkiye

Contents

1	Executive Summary	2
2	System Design Summary	4
2.1	Hardware Architecture	4
2.2	Software Architecture	4
3	Product Technical Specifications	5
3.1	System-Level Specifications	5
3.2	Edge Node Specifications	5
3.3	Gateway Specifications	5
3.4	Cloud and Mobile Application Specifications	5
4	Components Used and Cost Analysis	6
4.1	Edge Node Components	6
4.2	Gateway Components	6
5	Work and Task Distribution	7
6	Professional and Ethical Issues	7
6.1	Software Intellectual Property Considerations	7
6.1.1	Edge Node Software	7
6.1.2	Gateway Software	8
6.1.3	Mobile App and Cloud Software	8
6.2	Hardware Intellectual Property Considerations	8
6.3	Ethical Considerations	9
7	Conclusions	9

1 Executive Summary

Agricultural areas are frequently exposed to wildlife intrusions that may cause significant economic losses, threaten livestock safety, and damage property such as gates, fences, and corrals. Traditional countermeasures, including static fencing or continuous human monitoring, are often insufficient due to limited coverage, delayed response, and high operational costs. Therefore, there is a clear need for an autonomous, reliable, and real-time monitoring system that can detect wildlife activity, notify users promptly, and provide immediate deterrence without requiring constant human supervision.

FarmGate Sentinel is designed as a vision-first Internet of Things (IoT) system to address this need. The proposed solution consists of distributed camera-equipped edge nodes deployed near critical farm entry points, a central gateway unit, cloud-based storage and alerting services, and a mobile application for user interaction. Edge nodes continuously monitor their surroundings using a camera and auxiliary sensors, capturing images at a predefined rate and increasing capture frequency when motion is detected. The collected data are transmitted over a local wireless connection to the gateway, which performs real-time artificial intelligence-based animal detection. When a potential threat is identified, the system immediately sends photos to cloud which directly goes to mobile app and after alert decision of user system runs alarm as deterrence activity.

The system architecture is intentionally distributed to ensure robustness and scalability. Edge nodes are responsible for sensing and initial data acquisition, while computationally intensive tasks such as image processing is executed at the gateway using a trained deep-learning model. The gateway also provides local buffering capabilities to maintain operation during intermittent wide-area network outages. Cloud services enable persistent storage of incident data, role-based user access, and real-time synchronization with mobile clients, allowing users to monitor events remotely and take corrective actions when necessary. Also, through IMU sensor, stability of edge node can be tracked inside mobile app. PIR sensor provides movement activity which is also displayed inside mobile app. Reed Switch is used to track if door is open or closed.

Throughout the semester, the FarmGate Sentinel project progressed from system-level requirements to a fully integrated and demonstrable prototype. The team successfully implemented multi-node communication, real-time wildlife detection, automated alert generation, and a functional mobile application interface. The final system demonstrates reliable operation under realistic conditions, validates the feasibility of edge-assisted AI monitoring for agricultural environments, and highlights the effectiveness of integrating embedded systems, wireless communication, and machine learning into a cohesive engineering solution.

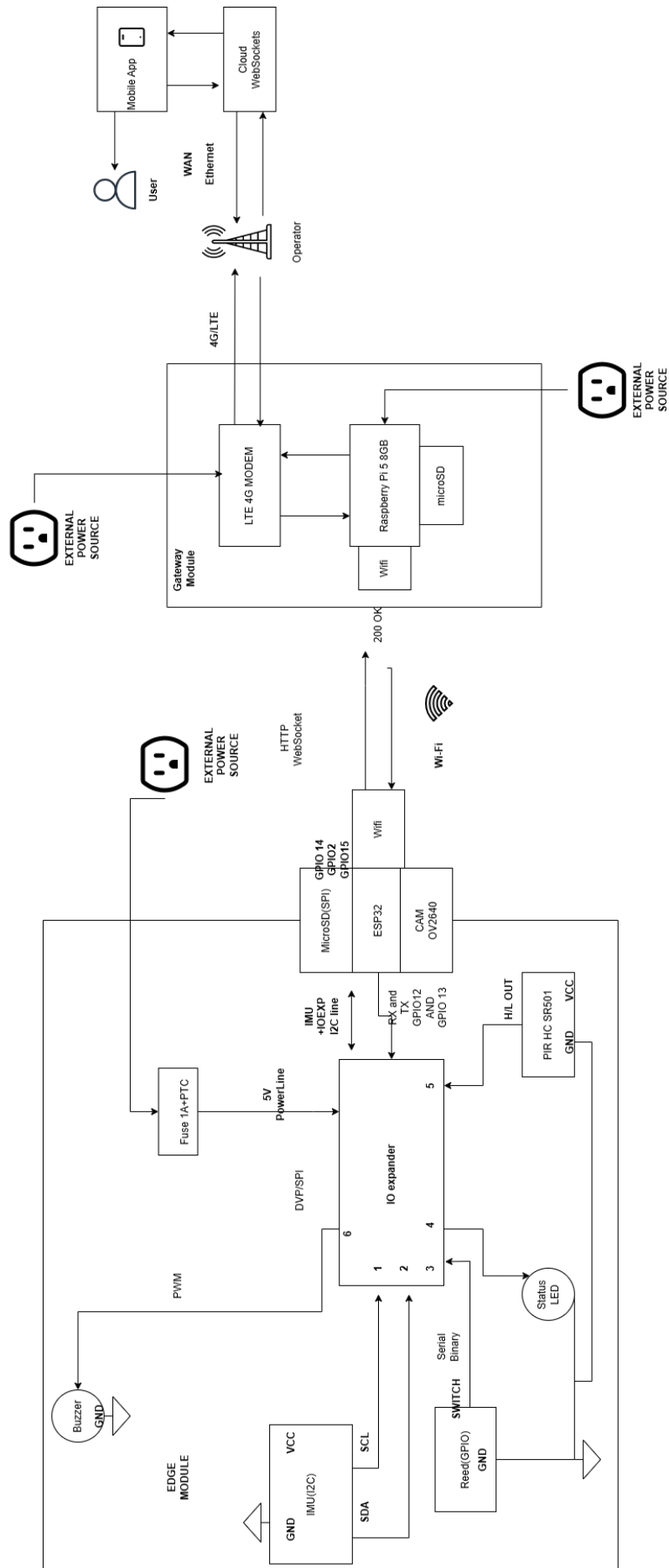


Figure 1: Overall system architecture block diagram [1].

2 System Design Summary

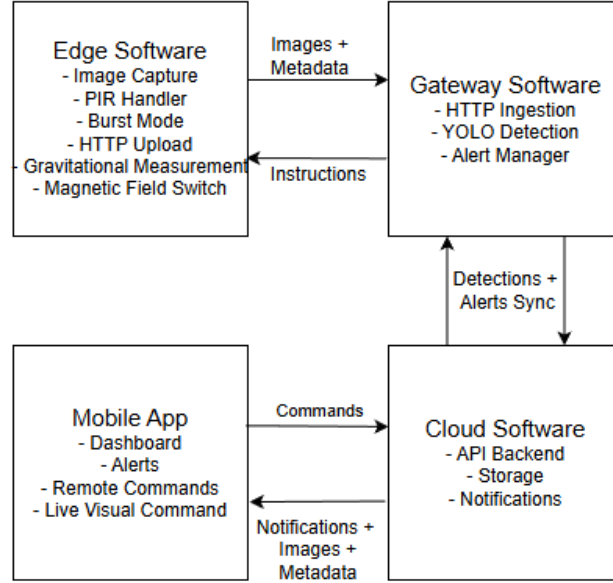


Figure 2: Overall software block diagram [1].

2.1 Hardware Architecture

FarmGate Sentinel is structured as a distributed hardware system composed of camera-equipped edge nodes, a central gateway, cloud services, and a mobile client. As illustrated in Figure 1, edge nodes are deployed in the field and communicate wirelessly with the gateway over a local Wi-Fi network. The gateway aggregates data from multiple nodes and provides wide-area connectivity to the cloud via an LTE interface. Users interact with the system through a mobile application connected to the cloud backend.

Each edge node is built around an ESP32-CAM platform and integrates a camera, motion and displacement sensors, and an audible actuator. The gateway is implemented using a Raspberry Pi 5, which performs data aggregation, buffering, and AI-based image processing. This modular hardware partitioning improves scalability and allows multiple edge nodes to operate under a single gateway.

2.2 Software Architecture

The software design follows a layered and distributed architecture, as shown in Figure 2. Edge node firmware is responsible for image acquisition, sensor monitoring, and communication with the gateway. The gateway software handles image reception, local buffering, and real-time animal detection using a deep-learning model.

Detection results and metadata are forwarded to the cloud, which provides persistent storage, user authentication, and synchronization with mobile clients. The mobile application presents alerts, system status, and control options to the user. This separation of responsibilities between edge, gateway, cloud, and mobile layers ensures robustness, maintainability, and efficient use of computational resources.

3 Product Technical Specifications

3.1 System-Level Specifications

The FarmGate Sentinel system is designed to support multiple edge nodes operating under a single gateway. The system processes image-based detections with end-to-end alert delivery to mobile clients within 30 s (85th percentile) from gateway detection. The gateway is capable of buffering captured data locally during wide-area network outages and successfully draining buffered data with at least 85% success upon network restoration [2]. Incident data are retained in cloud storage for a minimum duration of seven days. If user reaches data in mobile app, data downloaded once to mobile app and stored in local memory until its deleted. Role-based access control is enforced at the application level, where unauthorized operations result in HTTP 403 responses and corresponding audit logs.

3.2 Edge Node Specifications

Each edge node captures images at a baseline rate of 10 frames per minute with a tolerance of ± 1 frame per minute and supports 20 frame per minute when burst mode is active. A burst capture mode is activated upon user activates burst mode inside mobile app, increasing the capture rate temporarily.

The camera operates at a resolution of 1280×1024 pixels (SXGA), and each captured frame is transmitted as a JPEG image. Edge nodes are capable of detecting objects at a minimum distance of 1 m to 7 m under laboratory conditions, which corresponds to scaled detection for full-size animals in field deployment.

Wireless communication between edge nodes and the gateway is provided via 2.4 GHz Wi-Fi. A reliable connection is maintained in 10 m. Capture-to-gateway receipt latency is limited to 2.0 s (85th percentile) for SXGA images.

The edge node includes an audible buzzer actuator that is triggered upon gateway command. The buzzer activation duration is 30 s. The node also provides visual status indication via an LED to reflect operational states such as network joining, online operation, and burst capture.

PIR, IMU and Reed switch data send to gateway in 6 seconds blocks.

The edge node operates from a 5 V DC supply. Average current consumption is around 210 mA. Reverse polarity protection and overcurrent protection are implemented to prevent permanent hardware damage.

3.3 Gateway Specifications

The gateway is implemented using a Raspberry Pi 5 and processes incoming image streams from multiple edge nodes. Raspberry Pi is powered via 5V, 3A adapter. The detection processing latency from image receipt to incident proposal is limited to 2.0 s (85th percentile) for SXGA-resolution frames. Received sensor data is uploaded to cloud in every 6 seconds [2].

Local buffering at the gateway supports storage up to 1000 images during network disconnection. Upon restoration of the wide-area network, buffered data are forwarded to the cloud with a minimum success ratio of 85% within five minutes. Lats 10 photos from each node is stored inside memory of Raspberry Pi 5 to send cloud in case of live view request of user. The gateway operates as a Wi-Fi access point for edge nodes and uses an LTE-based uplink for wide-area communication [2].

3.4 Cloud and Mobile Application Specifications

The cloud infrastructure aggregates detection events into incidents when detections occur within a one-minute time window. For each incident, at least one thumbnail image and one original image are stored and retained for a minimum duration of seven days inside cloud.

Push notifications are delivered to registered mobile clients within 30 s (85th percentile) following gateway detection. The mobile application supports live image retrieval, where the most recent available image for a selected node is fetched and displayed within 10 s (95th percentile) [2].

Administrative users can modify system parameters such as capture rate, burst behavior, and actuator duration through the mobile interface. Configuration changes propagate to the gateway and edge nodes within 60 s of submission.

4 Components Used and Cost Analysis

In this part we will list the components used in the final product. We also bought some extra components for testing purposes outside of project component list which did not end up as a project's final component. These components will not be listed here. In the component list there are also equipments which are self bought and some components that are taken from the labs or from the instructor which are denoted as SB, L or I respectively.

4.1 Edge Node Components

Table 1 summarizes the primary components used in a single edge node.

Table 1: Edge Node Components

Component	Quantity	Unit Cost (TL)	Total Cost (TL)
ESP32-CAM Module (with Camera)	2	272.425	544.85
External Wi-Fi Antenna	2	48.37	96.74
Printed Circuit Board (PCB)	4(10cm*2cm)	20(SB)	80(SB)
Enclosure	2	140	280
PIR Motion Sensor	2	20.95	41.91
IMU Sensor Module	2	96.74	193.47
Reed Switch	2	16.93	33.86
MCP23017 DIP-28 I/O Expander	1	178.21	356.41
MicroSD Card 32GB	2	150	300
Glass Fuse 1A	2	5(SB)	10(SB)
1x40 Precision Female Headers	1	12.73	25.46
10k Ohm Resistor	2	L	L
4.7k Ohm Resistor	4	L	L
1uF Capacitor	4	L	L
0.1uF Capacitor	2	L	L
PTC 500mA	2	L	L
Circular Jack Adaptor	2	L	L
5V 1A Circular Jack AC-DC Adaptor	2	SB	SB
Total per Edge Node			1,962.7

4.2 Gateway Components

The gateway aggregates data from multiple edge nodes and performs real-time image processing. Table 2 lists the main components used in the gateway unit.

Table 2: Gateway Components and Cost

Component	Quantity	Unit Cost (TL)	Total Cost (TL)
Raspberry Pi 5 (8 GB RAM)	1	I	I
MicroSD Card (32 GB)	1	150	150
LTE Modem	1	1,777.90	1,777.90
Power Adapter (5 V, 3 A)	1	I	I
Enclosure	1	140	140
Total Gateway Cost			2,067.9
Total System Cost			4,030.6

5 Work and Task Distribution

- **Muhammed Gölcük:** Responsible for overall system design, high-level design. Responsible for the design and implementation of the edge node hardware and embedded firmware. His work included selecting and integrating the ESP32-CAM platform, camera module, sensors, and actuator components, as well as implementing sensor interfacing, image capture, and wireless communication routines on the edge node. He also performed electrical calculations, power measurements, and hardware-level testing. Responsible for non-functional design implementations, such as PCB implementation and enclosure design.[1]
- **Yusuf Semih Gürkök:** Responsible for the artificial intelligence and gateway processing subsystem. His contributions included dataset preparation, training and optimization of the YOLO-based animal detection model, and integration of the inference pipeline into the Raspberry Pi gateway. He also handled image preprocessing, postprocessing of detection results, and performance evaluation to balance detection accuracy and latency [1].
- **İsmail Özgenç:** Responsible for the cloud infrastructure and inter-system communication. He designed and implemented the communication layer between the gateway and the cloud, developed secure data transfer mechanisms, and managed data storage for images and metadata. His work ensured reliable synchronization between field devices and cloud services [1].
- **Zeynep İrem İpek:** Responsible for the mobile application development. She designed and implemented the user interface using the Flutter framework, integrated cloud APIs for alert retrieval and device control, and developed the notification and account management features. Her work enabled real-time user interaction with the FarmGate Sentinel system [1].

6 Professional and Ethical Issues

6.1 Software Intellectual Property Considerations

6.1.1 Edge Node Software

The edge node firmware is developed using the Arduino framework for ESP32 and relies entirely on open-source components that permit commercial deployment. Core ESP-IDF/Arduino SDK headers such as `esp_camera.h`, `esp_wifi.h`, `esp_sntp.h`, and `esp_http_server.h` are released under the Apache 2.0 license [3], which allows unrestricted commercial use. Networking and utility libraries including `WiFi.h`, `HTTPClient.h`, `SD_MMC.h`, and `time.h` are also

part of the Apache 2.0–licensed ESP32 Arduino core [3]. Image conversion utilities such as `img_converters.h` originate from the ESP-IDF camera driver and are likewise licensed under Apache 2.0 [3]. The C++ Standard Library features used on the edge node (e.g., `vector`, `memory`) follow the GCC libstdc++ GPL runtime exception [4], which permits commercial distribution of compiled firmware. Overall, the edge software stack is fully compliant with commercial licensing requirements and contains no copyleft components that impose disclosure of proprietary code.

6.1.2 Gateway Software

The Raspberry Pi gateway software uses open-source components that are all permitted for commercial use. The YOLOv11 model and the `ultralytics` Python package operate under the AGPL-3.0 license [5], which allows commercial deployment but requires that any modifications to YOLO-related source code be made publicly available. All other libraries in the inference and processing pipeline are permissively licensed for unrestricted commercial use, including PyTorch (BSD-3-Clause) [6], OpenCV-Python (Apache 2.0) [7], Pillow (HPND) [8], `tqdm` (MPL-2.0) [9], and `FiftyOne` (Apache 2.0) [10]. Standard Python modules used for file handling and networking—such as `os`, `pathlib`, `shutil`, and `glob`—are covered by the Python PSF License [11], which also supports commercial applications. Thus, the Raspberry Pi software environment complies fully with commercial licensing requirements, with AGPL obligations applying only to modified YOLO source components.

6.1.3 Mobile App and Cloud Software

The mobile application is developed using the Flutter framework and the Dart programming language, both of which are released under permissive licenses suitable for commercial use. The Flutter SDK—including its core UI toolkit, Material Design components, and the Maps SDK for Flutter used to provide geographical visualization and location services—is distributed under the BSD-3-Clause license [12], allowing modification and redistribution without copyleft restrictions. The Dart SDK and its standard libraries (including `io`, `async`, and `convert`) are similarly licensed under the BSD-3-Clause license [13], providing full commercial compatibility, while additional packages such as `path_provider` are published under the MIT license, which is likewise permissive and business-friendly. Android Studio, used for debugging and deployment, is distributed under the Apache License 2.0 with bundled third-party components [14], all of which permit commercial development. The Google Maps Platform, consumed through the Maps SDK for Flutter, operates under Google’s standard service terms and is subject to the Free Tier limit of 5,000 map loads per day [15, 16]. Therefore, the mobile software stack imposes no copyleft requirements and is fully compatible with commercial distribution of the FarmGate Sentinel application.

6.2 Hardware Intellectual Property Considerations

The hardware design of the FarmGate Sentinel system incorporates several commercially available off-the-shelf modules and components that were not originally designed by the project team. These components were selected to ensure reliability, availability, and rapid prototyping, while maintaining compliance with academic and potential commercial usage constraints.

The edge node hardware is centered around the ESP32-CAM module, which is manufactured by Espressif Systems and distributed as a commercially available development module. The ESP32 family of microcontrollers is provided with publicly available datasheets and reference designs that permit integration into custom hardware designs without restriction under standard commercial purchase terms [17]. The integrated OV2640 camera module is supplied by OmniVision and is similarly intended for integration into embedded vision systems without licensing limitations beyond standard component procurement [18].

Auxiliary hardware components used in the edge node, including the passive infrared (PIR) motion sensor, inertial measurement unit (IMU), reed switch, multiplexer, fuse, antenna, and microSD storage, are generic electronic modules sourced from multiple vendors. These components do not embed proprietary firmware or licensing constraints and are sold explicitly for integration into custom electronic products.

The gateway hardware is implemented using a Raspberry Pi 5 single-board computer. Raspberry Pi hardware platforms are distributed under permissive terms that allow use in educational, prototyping, and commercial products without restriction, subject only to standard compliance and certification requirements [19]. The LTE modem used for wide-area connectivity is a commercially available communication module intended for integration into embedded systems and does not impose intellectual property limitations on system-level design or software usage.

Overall, the hardware architecture of FarmGate Sentinel does not rely on proprietary or license-restricted subassemblies that would prevent commercial deployment. All third-party hardware components are used in accordance with their manufacturers' intended use cases, and the system-level hardware integration, PCB design, and enclosure design constitute original engineering work by the project team. Consequently, from a hardware intellectual property perspective, the FarmGate Sentinel system is fully compatible with both academic use and future commercialization.

6.3 Ethical Considerations

From an ethical perspective, FarmGate Sentinel was designed to minimize harm and respect user privacy. Although the system employs cameras, image capture is limited to specific monitored areas and is used solely for wildlife detection and security purposes. The system does not perform facial recognition or collect personally identifiable information.

Data storage and communication are handled using secure protocols to protect user data from unauthorized access. Role-based access control ensures that only authorized users can view incidents or issue control commands. Users are informed about the system's sensing capabilities and retain control over alerting and deterrence actions.

By considering intellectual property constraints, data privacy, and responsible system behavior, the FarmGate Sentinel project adheres to accepted professional and ethical standards in electrical and electronics engineering practice.

7 Conclusions

The FarmGate Sentinel project successfully achieved its primary objective of designing and implementing a functional, end-to-end wildlife monitoring and deterrence system for agricultural environments. Starting from high-level requirements, the team developed a complete solution that integrates embedded hardware, wireless communication, artificial intelligence, cloud services, and a mobile application into a cohesive and operational system.

A significant outcome of the project was the acquisition of new technical knowledge and practical experience. Team members gained hands-on expertise in ESP32-based embedded system development, camera interfacing, sensor integration, and low-power design considerations. The project also required learning and applying deep learning techniques for object detection, including dataset preparation, model training, and real-time inference optimization on resource-constrained hardware. In addition, the team developed experience with cloud synchronization, and mobile application development using modern cross-platform frameworks.

Beyond technical skills, the project strengthened the team's ability to manage a multidisciplinary engineering effort. Integrating hardware and software developed in parallel required careful coordination, systematic testing, and iterative refinement. The team gained experience

in system-level thinking, trade-off analysis, and debugging across multiple abstraction layers, which are essential skills for professional engineering practice.

Overall, the FarmGate Sentinel project provided valuable insight into the design and implementation of real-world IoT and AI-enabled systems. The knowledge gained throughout the semester, combined with the successful demonstration of a working prototype, represents a meaningful learning outcome of the EEE 495 course and forms a strong foundation for future academic or professional projects in embedded systems, artificial intelligence, and networked system design.

References

- [1] Y. S. Gürkök, M. Gölcük, İ. Özgenç, and Z. İ. İpek, *FarmGate Sentinel High-Level Design Document*, EEE495 Design II, Department of Electrical and Electronics Engineering, Bilkent University, Ankara, Türkiye, 2025.
- [2] Y. S. Gürkök, M. Gölcük, İ. Özgenç, and Z. İ. İpek, *FarmGate Sentinel Test Plan*, EEE495 Design II, Bilkent University, Ankara, Türkiye, 2025.
- [3] Espressif Systems, “ESP-IDF and Arduino-ESP32 License (Apache License 2.0),” 2024. [Online]. Available: <https://github.com/espressif/esp-idf/blob/master/LICENSE>
- [4] Free Software Foundation, “GCC Runtime Library Exception,” 2024. [Online]. Available: <https://www.gnu.org/licenses/gcc-exception-3.1.html>
- [5] Ultralytics, “Ultralytics YOLO License (AGPL-3.0),” 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics/blob/main/LICENSE>
- [6] PyTorch Foundation, “PyTorch License (BSD-3-Clause),” 2023. [Online]. Available: <https://github.com/pytorch/pytorch/blob/main/LICENSE>
- [7] OpenCV Developers, “OpenCV License (Apache License 2.0),” 2023. [Online]. Available: <https://github.com/opencv/opencv/blob/master/LICENSE>
- [8] Pillow Contributors, “Pillow License (HPND),” 2023. [Online]. Available: <https://github.com/python-pillow/Pillow/blob/main/LICENSE>
- [9] tqdm Developers, “tqdm License (MPL-2.0),” 2023. [Online]. Available: <https://github.com/tqdm/tqdm/blob/master/LICENSE>
- [10] Voxel51, “FiftyOne License (Apache License 2.0),” 2024. [Online]. Available: <https://github.com/voxel51/fiftyone/blob/develop/LICENSE>
- [11] Python Software Foundation, “Python PSF License,” 2024. [Online]. Available: <https://docs.python.org/3/license.html>
- [12] Flutter Authors, “Flutter SDK License (BSD-3-Clause),” 2024. [Online]. Available: <https://github.com/flutter/flutter/blob/master/LICENSE>
- [13] Dart Contributors, “Dart SDK License (BSD-3-Clause),” 2024. [Online]. Available: <https://github.com/dart-lang/sdk/blob/master/LICENSE>
- [14] JetBrains/Google, “Android Studio License (Apache License 2.0),” 2024. [Online]. Available: <https://developer.android.com/studio/terms>
- [15] Google LLC, “Google Maps Platform Terms of Service,” 2024. [Online]. Available: <https://cloud.google.com/maps-platform/terms>
- [16] Google LLC, “Google Maps Platform Pricing and Billing,” 2024. [Online]. Available: <https://cloud.google.com/maps-platform/pricing>
- [17] Espressif Systems, *ESP32 Series Datasheet*, Version 4.6, Espressif Systems, 2023. [Online]. Available: <https://www.espressif.com/>
- [18] OmniVision Technologies, *OV2640 CMOS Image Sensor Datasheet*, OmniVision Technologies, 2022. [Online]. Available: <https://www.ovt.com/>
- [19] Raspberry Pi Ltd., *Raspberry Pi 5 Product Brief*, Raspberry Pi Foundation, 2023. [Online]. Available: <https://www.raspberrypi.com/>